# Crimson: Collaborative Parameter Updates for Efficient Pipeline Training of Large Language Models

Yapeng Jiang
Sun Yat-sen University
Guangzhou, China

Wuhui Chen*
Sun Yat-sen University
Guangzhou, China

Ganhong Huang
Sun Yat-sen University
Guangzhou, China

Yuzhou Huang
Sun Yat-sen University
Guangzhou, China

Zicong Hong
Hong Kong University of Science and Technology
Hong Kong, China

Song Guo
Hong Kong University of Science and Technology
Hong Kong, China

Yue Yu
Peng Cheng Laboratory
Shenzhen, China

## Abstract

Large language models (LLMs) have driven significant progress in natural language processing, yet their training and fine-tuning remain limited by memory constraints, particularly the substantial memory footprints of optimizer states. Existing solutions address this challenge by offloading optimizer states and update tasks to the CPU, but this often leads to increased GPU idleness due to the CPU's limited computational capabilities, especially in pipeline parallelism.

To address this, we present Crimson, an efficient system that optimizes parameter updates in pipeline parallelism. Crimson employs a collaborative parameter updates scheme that distributes each stage's updates across its local GPU and CPU as well as the GPUs of subsequent stages. In addition, we design a multi-stage collaborative optimizer to coordinate computations and communications, together with a trapezoid bubble aware scheduling that formulates the process as an optimization problem to determine the optimal parameter update strategy. This approach maximizes GPU computational and memory efficiency. Experimental results across diverse model scales, parallelism configurations and state-of-the-art pipeline schedulings demonstrate that Crimson achieves throughput improvements exceeding 1.35× with ZeRO-Offload and ZeRO-Offload++, and up to 1.33× with Recomputation, along with increased GPU utilization.

*Corresponding author. Email: chenwuh@mail.sysu.edu.cn


*EuroSys '26, Edinburgh, Scotland, UK*

ACM ISBN 979-8-4007-2212-7/26/04
https://doi.org/10.1145/3767295.XXXXXXXX

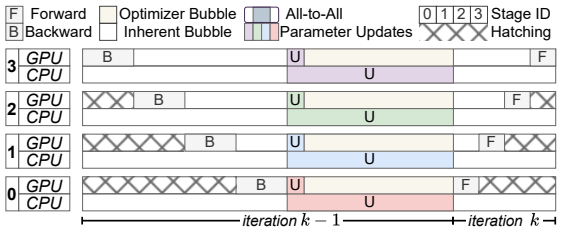
**ACM Reference Format:**
Yapeng Jiang, Wuhui Chen, Ganhong Huang, Yuzhou Huang, Zicong Hong, Song Guo, and Yue Yu. 2026. Crimson: Collaborative Parameter Updates for Efficient Pipeline Training of Large Language Models. In *Proceedings of 21st European Conference on Computer Systems (EuroSys '26) (EuroSys '26).* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3767295.XXXXXXXX

## 1 Introduction

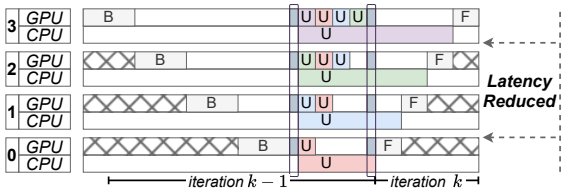
Large language models (LLMs) have scaled from billions to trillions of parameters, driving significant advances in natural language processing [2, 12]. Fine-tuning pre-trained LLMs enables small and medium-sized enterprises (SMEs) to efficiently adapt these models to domain-specific tasks with relatively short training. However, the memory wall remains a major bottleneck for scaling LLMs. For instance, fine-tuning a 70B-parameter LLM requires at least 1,260 GB of GPU memory to store model states [15], in addition to the memory required for activations. This typically demands four to eight training nodes, each equipped with eight 80 GB GPUs, which poses a substantial challenge for SMEs.

As a result, many exsiting research [9, 16, 18, 28] has focused on reducing GPU memory footprints during training. In settings with limited GPUs or extremely large models, optimizer states are the primary contributors to memory usage, requiring up to six times more memory than model weights when using the widely adopted Adam optimizer [14]. The state-of-the-art strategy for reducing optimizer memory usage is optimizer offloading [6, 17, 34, 42], which transfers all or part of the optimizer states and update tasks to the CPU, leveraging the CPU's computational and memory capabilities to support LLMs training.

Despite its memory advantages, optimizer offloading often increases GPU idle time due to the limited computational capabilities of CPUs. When combined with pipeline parallelism [5, 10, 22, 23, 29]—a widely adopted technique for training LLMs that distributes model layers across GPUs for parallel execution—substantial idle periods can occur. These delays, represented as pale *bubble* in Figure 1a, arise when GPUs must wait for parameter updates performed on CPUs. For example, fine-tuning a 50B-parameter [7] model




**(a)** Individual Parameter Updates




**(b)** Collaborative Parameter Updates

**Figure 1.** Two types of parameter updates workflows in pipeline parallelism. Colored "U" markers denote parameter updates at different stages, while pale bubbles indicate GPU idle periods during these updates.

on 32 H800 GPUs with pipeline parallelism and optimizer offloading results in a throughput reduction of 20–35%.

To optimize GPU computational efficiency while preserving memory savings through optimizer offloading in pipeline parallelism, we find that collaboratively updating parameters across stages—allowing later stages to assist earlier ones—is effective under the state-of-the-art schedulings such as 1F1B [5], Interleaved 1F1B [23], and Zero Bubble [29]. This is motivated by two key observations: **1) Later stages present lower GPU memory utilization.** As shown in Figure 1a, during the next iteration of 1F1B scheduling (as an illustrative example), different stages handle varying numbers of micro-batch forward. This leads to imbalanced memory usage [13, 48], where the GPU at stage $i$ (out of $s$ stages) must store activations for up to $s - i$ micro-batches, resulting in lower memory usage at later stages. **2) Later stages suffer longer GPU idleness.** Also shown in Figure 1a, data dependencies during the first forward lead to lagged computation in the next iteration. Specifically, stage $i$ must wait for stage $i - 1$ to complete its forward before starting its own, leading to prolonged GPU idleness at later stages.

Building on the two observations, we propose a new parameter update scheme called *Collaborative Parameter Updates*, illustrated in Figure 1b. The core concept is to allocate optimizer states and distribute corresponding update tasks not only to the CPU and the current stage's GPU but also to the GPUs of subsequent stages. This approach contrasts quietly with the traditional *Individual Parameter Updates* scheme, shown in Figure 1a, where optimizer states and update tasks are handled independently within each stage. For instance, in Figure 1b, Stage 0 distributes its parameter updates across its own GPU and CPU, as well as the GPUs of Stages 1, 2, and 3, using all-to-all communication to scatter gradients and gather updated parameters.

Although collaborative parameter updates can accelerate the parameter updates, applying it presents two collaborative challenges: **1) how to coordinate update tasks and corresponding communications.** Collaborative parameter updates requires two additional all-to-all communications: gradient scattering and updated parameter gathering. These operations involve heavy, synchronized communication across stages and can lead to GPU idleness. Specifically, earlier stages may idle while waiting to gather updated parameters, and later stages may also experience idleness due to data dependencies inherent in pipeline parallelism. Moreover, these all-to-all communications are sparse—each stage sends and receives quite different amounts of data—yet synchronization forces all stages to wait for the largest send or receive operation to complete, resulting in communication inefficiencies. **2) how to efficiently partition the update tasks.** Poor task distribution can lead to computational bottlenecks. If a stage is overloaded with update tasks, subsequent stages must wait to receive activations, while preceding stages are delayed in sending activations. Furthermore, fully leveraging the computational, memory, and communication resources of both GPUs and CPUs across all stages remains challenging, as coordinating these heterogeneous resources for optimal collaborative parameter updates is inherently complex.

To address these challenges, we propose Crimson, an efficient pipeline training system based on collaborative parameter updates, including the following core components:

**Multi-Stage Collaborative Optimizer.** This optimizer effectively exploits the idle periods between the last micro-batch's backward and the first micro-batch's forward in the next iteration at each stage. It decouples heavy communications, reorders parameter updates computations, overlaps them wherever possible, and dynamically manages parameter updates through GPU–CPU offloading. These designs collectively reduce overhead and improve the efficiency of collaborative parameter updates.

**Trapezoid Bubble Aware Scheduling.** This scheduling leverages the features of the bubble in the parameter update phase, formulating the optimal parameter updates strategy as a mixed-integer linear programming (MILP) problem. Through comprehensive profiling and solving this MILP problem, Crimson identifies the optimal parameter updates strategy, thereby improving overall GPU utilization.

The primary contributions of Crimson are as follows:

- We design a Multi-Stage Collaborative Optimizer that enables collaborative parameter updates, thereby accelerating the parameter updates process.
- We analyze collaborative parameter updates and introduce a model, supported by profiling data and a MILP formulation, to maximize their benefits.

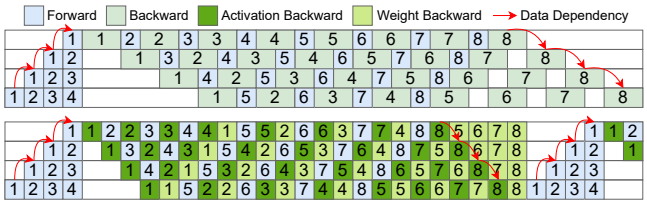



**Figure 2.** Two state-of-the-art pipeline schedulings: 1F1B (top) and Zero Bubble (bottom), highlighting inherent data dependencies in forward and backward.

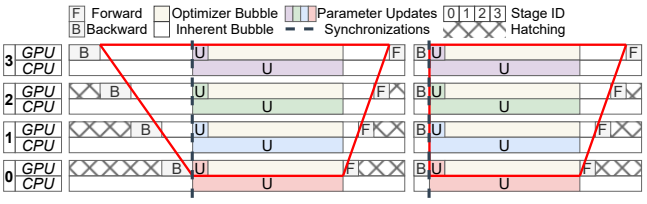



**Figure 3.** Optimizer offloading with (Interleaved) 1F1B (left) and Zero Bubble (right), highlighting the trapezoid bubble (optimizer and inherent bubbles) outlined in red, along with all-reduce synchronizations.

- We implement Crimson on top of Megatron-LM [15, 23, 36] and PyTorch [26], achieving throughput gains of more than 1.35× with ZeRO-Offload and ZeRO-Offload++, and up to 1.33× with Recomputation under state-of-the-art pipeline schedulings.

## 2 Background

### 2.1 Pipeline Parallelism

Pipeline parallelism [5, 10, 22, 23, 29] is a distributed training technique that accelerates large-scale model training, particularly for LLMs, across multiple GPUs or machines. The model is divided into stages, with each stage comprising consecutive layers executed on different GPUs. Input data flows sequentially through these stages during the forward, backward, and parameter updates, enabling highly concurrent training across multiple micro-batches.

GPipe [10] introduced a classic scheduling with a synchronization barrier, where all forward must complete before any backward begin. Although simple, this approach requires storing numerous micro-batch activations for backward, substantially increasing memory usage, especially in LLMs.

DAPPLE [5] proposed the 1F1B scheduling, which reduces memory pressure by initiating the backward immediately after the forward for each micro-batch. This execution proceeds through three phases: warmup (before the backward of the first micro-batch), steady, and cooldown (after the forward of the last micro-batch), as illustrated in Figure 2.

Megatron-LM [23] further introduced interleaved 1F1B scheduling, which partitions the computation graph into multiple subgraphs and assigns more than one subgraph to each stage. This approach reduces bubble but incurs additional communication overhead and higher memory usage.

Zero Bubble scheduling [29] splits the backward into activation backward and weight backward. Since the weight backward is independent across stages, it can be delayed while adjusting the activation backward, thus reducing the bubble, as shown in Figure 2.

### 2.2 Memory Footprints of LLMs Training

In mixed-precision training [21, 27], the memory footprint of LLMs can be categorized into two main components: model data and non-model data. Model data includes parameters (typically stored in 16-bit), gradients (stored in 16-bit or in 32-bit when gradient accumulation is used in pipeline parallelism), and optimizer states (stored in 32-bit), all of which must be retained for extended periods. Non-model data primarily consists of intermediate tensors (mainly stored in 16-bit), such as activations and temporary tensors, which are required for gradient computation but are released after use. Among model data, optimizer states—comprising master parameters and auxiliary variables necessary for parameter updates—are particularly critical for ensuring model convergence. The relative memory footprint of parameters, gradients, and optimizer states depends on the choice of optimizer; for example, Adam [14], the most widely used optimizer for LLMs training, exhibits a ratio of 1:2:6.

### 2.3 Optimizer Offloading

Optimizer offloading reduces GPU memory overhead by transferring some or all optimizer states and their associated update tasks to the CPU. ZeRO-Offload [34] first introduced offloading the entire parameter update process to the CPU. Subsequent works [6, 16, 17, 42] have investigated partial offloading. CPU-based optimizers employ techniques such as SIMD, loop unrolling, and multi-threading to accelerate tensor operations; however, they still perform significantly slower than GPU-based optimizers, resulting in prolonged parameter updates as well as GPU idleness.

As illustrated in Figure 3, integrating optimizer offloading with pipeline parallelism increases GPU idle time during parameter updates. This idle time, referred to as the *optimizer bubble*, combines with the inherent idle periods before and after updates, forming an extended idle region that we term the *trapezoid bubble* (highlighted in red). Moreover, pipeline parallelism requires all-reduce synchronizations prior to updates to ensure the integrity of 16-bit gradients [21, 25, 29], such as global checks for NaN and INF values are performed [21].

## 3 Motivation

To optimize GPU utilization while preserving the memory savings of optimizer offloading under the limited GPU resources typical of SMEs, we examine key insights into memory usage, computation, and communication across stages.

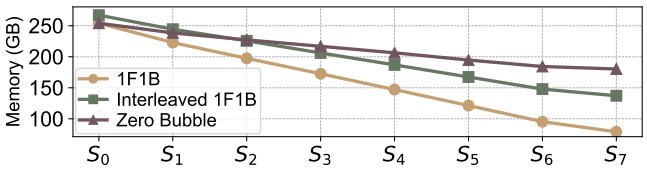



**Figure 4.** Imbalanced memory footprints across stages ($S_0$ to $S_7$ denote stage IDs) in pipeline parallelism.

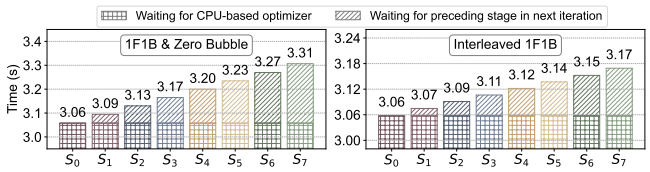



**Figure 5.** GPU idleness following parameter updates, including wait time for CPU updates and lag in next iteration.

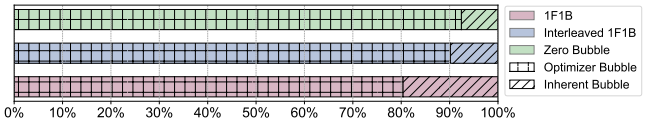



**Figure 6.** Comparison of bubbles caused by CPU-based optimizers with those inherent in different schedulings.

### 3.1 Memory Footprints across Stages

In 1F1B scheduling, the warmup phase requires each stage to perform multiple forward and store activations in GPU memory. During the steady phase, stages alternate between forward and backward, releasing and storing activations, while the cooldown phase consists solely of backward to clear remaining activations. Peak memory usage occurs at the end of warmup: in a pipeline with $s$ stages, the first stage holds $s$ micro-batches activations, while subsequent stages hold fewer, with the last stage storing only one. Interleaved 1F1B and Zero Bubble exhibit a similar trend, with activation storage decreasing progressively from the first stage to the last, although their schedulings differ from 1F1B.

Figure 4 illustrates the memory footprints across stages when training a 50B Llama 3 [7] model (a variant of Llama 3 70B) using full optimizer offloading (ZeRO-Offload) on 32 H800 GPUs, configured with tensor parallelism [36] of 4 and pipeline parallelism of 8 under 1F1B, Interleaved 1F1B, and Zero Bubble schedulings. Despite the additional embedding layer in the first stage and the head layer in the last, memory usage across adjacent stages follows a linear decreasing trend. Under these configurations, the first stage consumes 3.21×, 1.95×, and 1.41× more memory than the last stage for 1F1B, Interleaved 1F1B, and Zero Bubble schedulings, respectively, leaving substantial GPU memory unused in later stages.

**Insight 1: *Later stages present lower GPU memory utilization.*** *Many state-of-the-art schedulings result in imbalanced memory footprints across stages, leaving a considerable portion of GPU memory underutilized in later stages.*

### 3.2 GPU Idleness in Parameter Updates

Within the trapezoid bubble shown in Figure 3, all stages begin update tasks after the all-reduce synchronizations. However, they remain idle for extended periods while waiting for the CPU to finish parameter updates. In the following iteration, the startup time of the first micro-batch's forward differs across stages due to data dependencies in pipeline parallelism: stage $i$ must wait for the output of stage $i-1$ for the same micro-batch, leading to noticeable lag.

Figure 5 shows GPU idle time after completing updates under full optimizer offloading and various schedulings, based on the same experiments described in subsection 3.1. The idleness difference between adjacent stages is approximately equal to the forward time of a single micro-batch, causing the last stage to experience substantially more idle time than the first. Figure 6 further compares optimizer bubble with other inherent bubble in different pipeline schedulings, showing that the optimizer bubble is 4.1×, 9.2×, and 12.2× larger than all other bubbles in 1F1B, Interleaved 1F1B, and Zero Bubble, respectively. These results clearly demonstrate that optimizer offloading substantially increases GPU idleness across schedulings, due to the limited computational capabilites of CPUs, as discussed in subsection 2.3.

**Insight 2: *Later stages suffer longer GPU idleness.*** *Compared with earlier stages, later stages remain idle for longer intervals between completing update tasks and starting the next forward due to data dependency.*

### 3.3 Modern Inter-GPU Connections

Advances in GPU interconnect technologies have dramatically increased communication bandwidth within and across training nodes. For example, NVIDIA H100 servers equipped with fourth-generation NVLink support bidirectional transfer rates of up to 900 GB/s [24]. Fifth-generation PCIe links among GPUs deliver up to 128 GB/s of bidirectional bandwidth [45], while NDR InfiniBand enables inter-node communication speeds of up to 400 Gb/s [44].

**Insight 3:** *High-bandwidth inter-GPU connections enhance the communication capabilities among GPUs.*

### 3.4 Collaborative Parameter Updates

These insights motivate our proposed scheme, *Collaborative Parameter Updates*, which allocates optimizer states not only to the CPU and the current stage's GPU but also to GPUs in later stages. This design delegates corresponding update tasks to later stages, thereby leveraging their underutilized memory and computational resources.

Specifically, optimizer states for each stage are partitioned and distributed across the CPU, the current stage GPU, and

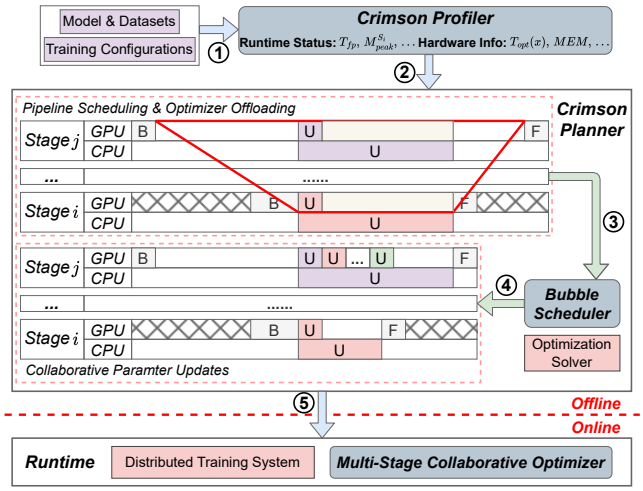



**Figure 7.** System Architecture of Crimson.

subsequent stage GPUs. After gradient clipping and synchronizations, earlier stages transfer gradients both to the CPU and to the designated later stages, enabling collaborative execution of update tasks. Once updates are completed, each stage synchronizes updated parameters back from the CPU and the contributing later stages. After the collaborative updates for a stage are finalized, the system proceeds with the forward of the next iteration for the stage.

## 4 The Crimson System Overview

Crimson organizes its workflow into steps ①–⑤ in Figure 7. Users first provide the dataset, model, and training configurations. Crimson then executes several training iterations with optimizer offloading to collect runtime and hardware information through the Profiler (①). The profiling data is analyzed by the Crimson Planner, which evaluates the trapezoid bubble (②) using the Bubble Scheduler. Based on this evaluation, the Bubble Scheduler develops a parameter update strategy designed to maximize the efficiency of collaborative parameter updates (③ and ④). This strategy is subsequently integrated into the runtime, where the multi-stage collaborative optimizer enables the collaborative scheme (⑤). Through steps ①–⑤, Crimson establishes its parameter update strategy, facilitating efficient pipeline training.

**Offline Overheads.** Both the Profiler and Planner operate offline. The Profiler collects data during the initial training iterations, while the Planner uses this data to generate an optimal parameter update strategy (detailed in section 6), which is then passed to the online component. In our evaluations, the offline phase typically required only 3–5 minutes, a negligible cost relative to the overall training process. The online component consists of the distributed training system, responsible for core training and parallelism, and the multi-stage collaborative optimizer, which enables collaborative parameter updates (detailed in section 5).

## 5 Multi-Stage Collaborative Optimizer

In this section, we first analyze the importance of coordinating computations and communications within the proposed scheme. We then introduce designs in the multi-stage collaborative optimizer to address one of the central collaboration challenge: **how to coordinate update tasks and corresponding communications**.

### 5.1 Challenge

Collaborative parameter updates require two additional communication steps: gradient scattering (each stage receives a subset of gradients from preceding stages and sends another subset to subsequent stages) and updated parameter gathering (the reverse process). These steps, combined with the out-of-order execution of update tasks, are naturally implemented using all-to-all synchronized collective communication. However, this approach suffers from low communication efficiency for two main reasons.

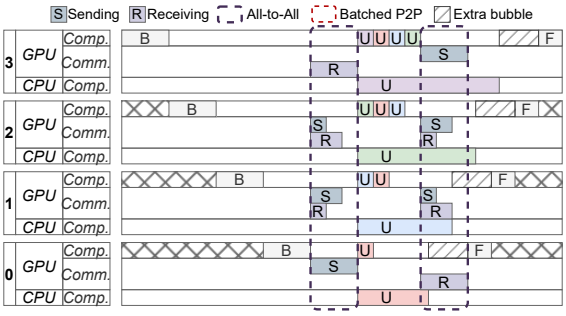
First, all-to-all communication is a heavy synchronized operation across stages, often introducing additional GPU idle time. As shown in Figure 8a, the two all-to-all operations span long durations across stages, particularly because pipeline parallelism often extends across multiple nodes. In the case of updated parameters gathering, this can create additional idle periods (bubbles) on GPUs: earlier stages must wait for the gathering to complete before initiating the forward in the next iteration, while later stages must wait for activations from earlier stages to proceed, further delaying computation. Second, the two all-to-all operations are inherently sparse. Even if each stage requires assistance from all subsequent stages and update tasks are evenly distributed, as illustrated in Figure 8a, communication directions and volumes vary significantly across stages. This sparsity forces all stages to wait for the slowest send or receive operation, reducing overall communication efficiency.

**Challenge 1:** *How can update tasks and their corresponding communications be effectively coordinated?*

### 5.2 Orchestration of Comm. and Comp.

To address Challenge 1, we propose **Decomposition of Scattering-Gathering** and **Reordering of Computation Sequences** to orchestrate communications and computations to ensure efficient coordination between them.

In pipeline schedules with inherent bubbles preceding parameter updates—such as (Interleaved) 1F1B—all stages except the first experience GPU idle time following the last micro-batch's backward. For stage $i$ out of $s$ total stages, this idle period is approximately $(i-1)$ times the duration of a single micro-batch's backward, as illustrated prior to the gradient scattering phase in Figure 8a. Although this idle time cannot be used for subsequent parameter updates computations, it can be exploited to initiate most of the gradient




**(a)** Uncoordinated Communications and Computations

**(b)** Decomposed Communications and Coordinated Computations

**Figure 8.** Two coordination ways for communications and computations. "Comm." and "Comp." denote the communication and computation at each stage, respectively.

scattering early. For stage $i$ ($0 < i < s$), gradients are prepared for transferring to subsequent stages, while buffers are allocated for receiving gradients from preceding stages. We employ batched point-to-point (p2p) communication, allowing each stage to begin scattering gradients immediately upon completing its backward, while inserting receive operations for expected incoming gradients. For the first stage, scattering is also performed via batched p2p, thereby eliminating the need for all-to-all communication.

In most schedules with inherent bubbles following parameter updates—such as (Interleaved) 1F1B and Zero Bubble—we further decompose the all-to-all communication required for updated parameter gathering by reordering the computation sequence of parameter updates. To ensure that earlier stages receive updated parameters as soon as possible and can initiate the next iteration earlier, we sort the update computations on later stages and begin transferring parameters immediately after each computation completes. This allows each stage to use batched p2p to receive updated parameters significantly earlier, enabling faster initiation of the next iteration compared to a synchronized all-to-all. This reordering not only eliminates the need for all-to-all communication in parameter gathering but also overlaps communication with computation, further improving efficiency.

As shown in Figure 8b, the proposed decomposition and reordering transform all-to-all communication into multiple batched p2p operations. This enables gradient scattering to occur during previously idle GPU periods and allows subsequent computations to overlap with parameter gathering,

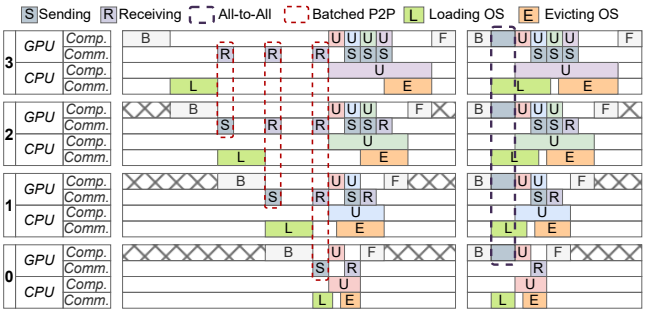



**Figure 9.** Final communication and computation workflow of the multi-stage collaborative optimizer under (Interleaved) 1F1B (left) and Zero Bubble (right) schedulings.

as indicated by the red dashed blocks. Consequently, these approaches mitigate communication inefficiencies and eliminate additional bubbles caused by all-to-all communication in collaborative parameter updates.

### 5.3 Design Reinforcement

To further reduce CPU workload during parameter updates and exploit idle periods preceding them, we reinforce our design by enhancing the orchestration of communication and computation at each stage.

Specifically, once the backward of the last micro-batch completes, all activations of the $s - i$ micro-batches are released, making GPU memory at stage $i$ more available. Leveraging this increased memory availability—along with the idle GPU-to-CPU transfer channel—we propose **Dynamic Loading and Evicting** of optimizer states at each stage. As illustrated in Figure 9, after completing the final backward, each stage not only scatters its ready gradients but also loads a portion of optimizer states from the CPU. These states are then used to perform the corresponding parameter updates on the GPU. Once the updates are completed, the modified optimizer states are evicted back to the CPU.

For schedulings such as (Interleaved) 1F1B, loading operations are primarily inserted into the inherent bubbles that precede parameter updates. Similarly, in most schedulings, evicting operations are placed into bubbles inherent to pipeline execution or caused by CPU-based optimizer.

**Preventing PCIe Bottlenecks.** To avoid transfer bottlenecks, the workload transferred from the CPU to the GPU is carefully limited and scheduled according to the trapezoid bubble aware scheduling described in section 6.

### 5.4 Final Workflow

Parameter updates are partitioned according to the distribution of optimizer states, which determines the corresponding update tasks. Optimizer states are classified by their storage location and the device responsible for executing the updates: **1)** optimizer states stored in the CPU memory of stage $i$, denoted as $OS_{\text{cpu}}^{(i)}$, remain on the CPU and are updated there;

**Algorithm 1:** Workflow of the Multi-Stage Collaborative Optimizer at stage $i$ ($0 < i < s$). **Clipping** refers to gradient clipping, and **Opt.step** denotes the optimizer's step.

**Input:** $OS^{(i)}_{\text{cpu}}, OS^{(i)}_{\text{gpu}}, OS^{(i)}_{\text{c2g}}, G^{(i)}_{\text{cpu}}, G^{(i)}_{\text{gpu}}, G^{(i)}_{\text{c2g}}, P^{(i)}_{\text{cpu}}, P^{(i)}_{\text{gpu}}, P^{(i)}_{\text{c2g}}$

1 Initialize $G_{\text{send}}$, $G_{\text{recv}}$, $P_{\text{send}}$ and $P_{\text{recv}}$ as buffers lists.

/* ************ On GPU Side ************ */

2 load_optimizer_states($OS^{(i)}_{\text{c2g}}$) **and** offload_gradients($G^{(i)}_{\text{gpu}}$)

3 **for** $j = i + 1$ **to** $s - 1$ **do**

4 send_gradients(src=$i$, dest=$j$, $G_{\text{send}}[j]$)

5 **for** $j = 0$ **to** $i - 1$ **do**

6 receive_gradients(src=$j$, dest=$i$, $G_{\text{recv}}[j]$)

/* ——— Allreduce Synchronizations ——— */

7 **for** $j = 0$ **to** $i - 1$ **do**

8 **Clipping(**$G_{\text{recv}}[j]$**)**

9 $P_{\text{send}}[j]$ = **Opt.step(**$OS^{(i)}_{\text{gpu}}[j], G_{\text{recv}}[j]$**) and** send_parameters(src=$i$, dest=$j$, $P_{\text{send}}[j]$)

10 **Clipping(**$G^{(i)}_{\text{gpu}}$**) and Opt.step(**$OS^{(i)}_{\text{gpu}}[i], G^{(i)}_{\text{gpu}}$**)**

11 **Clipping(**$G^{(i)}_{\text{c2g}}$**) and Opt.step(**$OS^{(i)}_{\text{c2g}}, G^{(i)}_{\text{c2g}}$**)**

12 **for** $j = i + 1$ **to** $s - 1$ **do**

13 receive_parameters(src=$j$, dest=$i$, $P_{\text{recv}}[j]$)

14 evict_optimizer_states($OS^{(i)}_{\text{c2g}}$) **and** load_parameters($P^{(i)}_{\text{cpu}}$)

/* ************ On CPU Side ************ */

15 **Clipping(**$G^{(i)}_{\text{cpu}}$**) and Opt.step(**$OS^{(i)}_{\text{cpu}}, G^{(i)}_{\text{cpu}}$**)**

**2)** optimizer states residing in the GPU memory of stage $i$ (including those delegated from preceding stages), denoted as $OS^{(i)}_{\text{gpu}}$, are updated by the GPU; and **3)** optimizer states dynamically loaded from the CPU and evicted back after updates, denoted as $OS^{(i)}_{\text{c2g}}$, which reside in CPU memory during forward–backward but are temporarily transferred to the GPU for updates, executed on the GPU. Correspondingly, gradients and parameters at stage $i$ are partitioned into $G^{(i)}_{\text{cpu}}$, $G^{(i)}_{\text{gpu}}$, and $G^{(i)}_{\text{c2g}}$, as well as $P^{(i)}_{\text{cpu}}$, $P^{(i)}_{\text{gpu}}$, and $P^{(i)}_{\text{c2g}}$, respectively.

algorithm 1 details the distributed workflow of the multi-stage collaborative optimizer at stage $i$. Notably, the loading of $OS^{(i)}_{\text{c2g}}$ overlaps with the offloading of $G^{(i)}_{\text{cpu}}$ (line 2), exploiting the full-duplex capability of PCIe. Similarly, evicting and loading operations overlap in line 14. Batched p2p communications in lines 3–6 implement gradient scattering, while those and **Opt.Step** in lines 7–9 and 12–13 correspond to updated parameter gathering and corresponding parameter updates. These communication patterns result from the *decomposition of scattering–gathering*, and the execution order of update tasks in these lines reflects the *reordering of computation sequences*. Lines 10 and 15 perform parameter updates for $P^{(i)}_{\text{c2g}}$ and $P^{(i)}_{\text{cpu}}$, respectively. Finally, the comments "On GPU Side" and "On CPU Side" indicate the device executing subsequent operations, while "Allreduce Synchronizations" refer to the allreduce operations described in subsection 2.3.

**Table 1.** Variables used in the MILP formulation. Optimization variables define the search space, while auxiliary variables can be derived from the optimization variables.

| | |
|---|---|
| **Constant variables:** | |
| $\mathcal{S}$ | Set of stage IDs in pipeline parallelism $\{0, 1, \ldots, s-1\}$ |
| $\mathbb{P}^{(i)}_{\text{pp}}, \mathbb{OS}^{(i)}_{\text{pp}}$ | Parameters and optimizer states on stage $i$ |
| $\mathbf{M}^{(i)}_{\text{act}}$ | Profiled activation memory footprints on stage $i$ |
| $\tau, \mathbf{M}_{\text{total}}$ | Threshold and profiled GPU memory capacity |
| $\mathbf{T}^{(i)}_{\text{pre}}, \mathbf{T}^{(i)}_{\text{post}}$ | Profiled inherent bubble time before (pre) and after (post) the parameter updates |
| $\kappa_{\text{opt}}$ | Ratio of optimizer states to parameters |
| **Optimization variables:** | |
| $x_i$ | Non-negative integer; Optimizer states stored on the CPU of stage $i$ |
| $y_i$ | Non-negative integer; Optimizer states dynamically loaded and evicted on the GPU of stage $i$ |
| $z_{i,j}$ | Non-negative integer, $i \leq j$; Optimizer states placed on the GPU of stage $j$, delegated from stage $i$ |
| **Auxiliary variables:** | |
| $z^-_i, z^+_i$ | Optimizer states stored on subsequent stages (-) and held from preceding stages (+) at stage $i$ |
| $M^{(i)}_{\text{iter}}$ | Memory footprints at stage $i$ in a training iteration |
| $M^{(i)}_{\text{fp,bp}}, M^{(i)}_{\text{opt}}$ | Memory footprints at stage $i$ in forward-backward and parameter updates of a training iteration |
| $M(x)$ | Calculating the memory footprints from $x$ ($\propto x$) |
| $T_{\text{opt}}(x)$ | Calculating the time to update $x$ ($\propto x$) |
| $T_{\text{load}}(x)$ | Calculating the time to load $x$ to GPU ($\propto x$) |

# 6 Trapezoid Bubble Aware Scheduling

In this section, we leverage resource features during parameter updates to model collaborative parameter updates. To enhance the performance of this scheme, we introduce trapezoid bubble aware scheduling, which addresses a key challenge: **how to efficiently partition the update tasks**.

## 6.1 Challenge

First, an unreasonable distribution of update tasks can lead to computational bottlenecks. If a stage is overloaded, subsequent stages must wait to receive activations, while preceding stages are delayed in sending activations—causing all stages to stall in the next iteration. Second, due to the complex workflow of the multi-stage collaborative optimizer—which operates over various computational, memory, and communication resources across all stages—identifying the optimal parameter updates strategy is challenging.

**Challenge 2:** *How can update tasks be efficiently partitioned to achieve an optimal parameter updates strategy?*

## 6.2 System Modeling & Problem Formulation

To address Challenge 2, we focus on the most time-consuming operations and constrained resources when constructing constraints for the modeling. This analysis reveals several linear or approximately linear relationships involving CPU computation, GPU memory usage, and GPU–CPU data transfer.

Based on these relationships, we formulate the problem as a mixed-integer linear programming (MILP) model, defined in Equation 1, with all variables listed in Table 1.

Following the definitions in subsection 5.4, we specify the optimization variables as follows: $x \in \mathbb{Z}_{\geq 0}^{1\times s}$ represents all $OS_{\text{cpu}}^{(i)}$; $y \in \mathbb{Z}_{\geq 0}^{1\times s}$ represents all $OS_{\text{c2g}}^{(i)}$; and $z \in \mathbb{Z}_{\geq 0}^{s\times s}$ (for $i \leq j$) represents the core variables in collaborative parameter updates, where $z_{i,j}$ denotes the optimizer states originating from stage $i$ but stored on the GPU of stage $j$ (including $i = j$). Additionally, $\mathbb{P}_{\text{pp}}^{(i)}$ and $\mathbb{OS}_{\text{pp}}^{(i)}$, as defined in Table 1, denote the original parameters and optimizer states associated with stage $i$ after the parallelism configuration is established.

$$\textbf{min } T_{\text{opt}}(x_0) \tag{1}$$

$$\textbf{s.t. } x_i + y_i + \sum_{j=i}^{s-1} z_{i,j} = \mathbb{OS}_{\text{pp}}^{(i)},\ \forall i \in \mathcal{S}, \tag{2}$$

$$M_{\text{iter}}^{(i)} = \max(M_{\text{fp,bp}}^{(i)}, M_{\text{opt}}^{(i)}) \leq \tau\, \mathbf{M}_{\text{total}},\ \forall i \in \mathcal{S}, \tag{3}$$

$$0 \leq T_{\text{opt}}(x_i - x_{i-1}) \leq \mathbf{T}_{\text{post}}^{(i)} - \mathbf{T}_{\text{post}}^{(i-1)},\ \forall i \in \mathcal{S} \setminus \{0\}, \tag{4}$$

$$2T_{\text{load}}(y_i) \leq T_{\text{opt}}(x_0) + \mathbf{T}_{\text{pre}}^{(i)} + \mathbf{T}_{\text{post}}^{(i)},\ \forall i \in \mathcal{S}. \tag{5}$$

**Fundamental Constraints.** We impose the constraints in Equation 2 to ensure that the sum of partitioned optimizer states equals the total number of optimizer states.

Since GPUs provide significantly greater computational capabilities than CPUs but have limited memory, as well as the PCIe bandwidth is limited, additional constraints are required to capture GPU memory footprints, CPU computation time, and GPU–CPU transfer time.

$$z_i^- = \sum_{j=i+1}^{s-1} z_{ij},\ z_i^+ = \sum_{j=0}^{i-1} z_{ji} \tag{6}$$

$$M_{\text{fp,bp}}^{(i)} = 3M(\mathbb{P}_{\text{pp}}^{(i)}) + M(\mathbb{OS}_{\text{pp}}^{(i)}) + \mathbf{M}_{\text{act}}^{(i)} + M(-x_i - y_i - z_i^- + z_i^+) \tag{7}$$

$$M_{\text{opt}}^{(i)} = 3M(\mathbb{P}_{\text{pp}}^{(i)}) + M(\mathbb{OS}_{\text{pp}}^{(i)}) + M(-x_i - z_i^- + z_i^+) + \frac{2}{\kappa_{\text{opt}}} M(z_i^+) \tag{8}$$

**Constraints on GPU Memory Footprints.** The peak memory footprints during an training iteration may occur either in the forward–backward (for early stages) or in the optimizer step (for later stages, which must store delegated optimizer states from preceding stages). To capture this, we define auxiliary variables in Equation 7 and Equation 8. Specifically, $z_i^-$ and $z_i^+$ denote the optimizer states that stage $i$ delegates to subsequent stages and holds from preceding stages, respectively. $M_{\text{fp,bp}}^{(i)}$ accounts for memory used by parameters, gradients, activations, and optimizer states during the forward–backward, while $M_{\text{opt}}^{(i)}$ accounts for parameters, gradients, and optimizer states required during the optimizer step. Given the memory footprints ratio among parameters, gradients, and optimizer states is $1 : 2 : \kappa_{\text{opt}}$, we omit gradients and introduce a coefficient of 3 before $M(P_{\text{pp}}^{(i)})$, and $2/\kappa_{\text{opt}}$ before $M(z_i^+)$. To prevent out-of-memory (OOM) errors, we impose the GPU memory constraints in Equation 3 for all stages, where $\tau \in (0, 1)$ is a hyperparameter that reserves additional memory overheads for CUDA context, temporary tensors, and other GPU operations.

**Constraints on CPU Computation Time.** To avoid stalls from overloaded stages, we ensure that the increase in CPU computation time between adjacent stages is less than the forward time of a single micro-batch ($\mathbf{T}_{\text{post}}^{(i)} - \mathbf{T}_{\text{post}}^{(i-1)}$ in Equation 4). To leverage the monotonic property in *Insight 2*, this increase must also be non-negative. Thus, we impose the CPU computation time constraints in Equation 4 for all stages except the first. Since $T_{\text{load}}(x)$ is proportional to $x$, these constraints promote a nearly linear increase in CPU workloads from the first to the last stage.

**Constraints on GPU–CPU Transfer Time.** To prevent GPU–CPU transfers from becoming a bottleneck, we require the transfer time to remain within the combined duration of updating $x_0$ and the bubbles before and after the updates. Specifically, the total loading and evicting time must not exceed the trapezoid bubble duration at stage $i$, as discussed in subsection 5.3. Because the loading time is approximately equal to the evicting time of $y_i$, we impose the GPU–CPU communication constraints in Equation 5 for all stages.

**Objective Function.** The MILP formulation minimizes the update duration of $x_0$, thereby reducing the total parameter updates time across all stages, while satisfying all constraints and respecting data dependencies.

### 6.3 MILP Solver

Determining the optimal parameter update strategy for the multi-stage collaborative optimizer requires both profiling and solving the MILP. Profiling data—primarily hardware specifications and runtime metrics—is collected by executing first several training iterations with full optimizer offloading. And the hyperparameter $\tau$ is estimated by measuring additional GPU memory overheads in these iterations.

Notably, full optimizer offloading ($x_i = OS_{\text{pp}}^{(i)}, y_i = 0, z_{i,j} = 0$) represents a specific feasible solution to the MILP defined in Equation 1 and can serve as a fallback strategy. The MILP is stage-centric: its constraint complexity grows proportionally with $s^2$, where $s$ is the number of pipeline stages and is typically selected from $\{4, 8, 16, 32\}$ in practice, rather than directly with model parameter count. This allows the commercial solver to identify the optimal solution within a short time across different model scales under moderate stage counts. In our evaluations, the optimal solution is obtained in less than 1 minute, after which the optimization variables are exported as the parameter update strategy.

Since Crimson's parameter update strategy is derived from profiled update and transfer costs, its validity depends

on whether the underlying cost structure remains unchanged. Configuration changes that materially affect these costs require re-profiling and re-solving (e.g., micro-batch size, sequence length, or parallelism layout). In contrast, changes in global batch size realized only through gradient accumulation often preserve the same per-step update-phase cost structure, so an existing plan can often be reused.

# 7 Generality & Applicability Discussion

In the design of Crimson, the Multi-Stage Collaborative Optimizer is made compatible with different schedulings, including those with or without idleness before parameter updates. Its communication is confined to the parameter-update portion of each iteration (i.e., after gradient synchronization and before parameter synchronization in data-parallel) and is coordinated separately from forward/backward communications (e.g., tensor-parallel and pipeline-parallel communications), so Crimson does not introduce new communication dependencies on the main training path. Within this update phase, the Crimson plan schedules stage-to-stage parameter-update transfers together with local update execution, with overlap applied when possible. For Trapezoid Bubble Aware Scheduling, we abstract memory usage, bubble time across different schedulings, and the parameter $\kappa_{\text{opt}}$ to ensure compatibility with diverse optimizer algorithms. Overall, these designs and our evaluations demonstrate that Crimson generalizes effectively to a wide range of schedulings, parallelism combinations, and optimizer algorithms.

From a runtime perspective, Crimson also supports changing training configurations without requiring online optimization. To accommodate varying micro-batch sizes, global batch sizes, and gradient accumulation settings, Crimson can precompute a small set of plans for representative configurations and switch among them at runtime. Re-profiling/re-solving is only needed when a new setting falls outside this profiled set or materially changes the update-phase cost structure. In contrast, switching among precomputed plans is an infrequent control-path reconfiguration: it selects a plan and updates the associated runtime state for the target configuration (e.g., buffer allocations/bindings, and the residency/placement of optimizer states), without invoking any online optimization. The switching cost is incurred only at configuration-change boundaries (not per iteration) and is amortized over subsequent iterations.

In practice, Crimson is most beneficial when pipeline training uses optimizer offloading and the update phase introduces non-trivial stage-level idleness. Its gains diminish when most model parameters and optimizer states can remain on GPU (i.e., offloading becomes limited), or when update-phase costs are already fully hidden or otherwise cease to be a dominant bottleneck.

# 8 Implementation

We implement Crimson on top of PyTorch [26] and Megatron-LM [15, 23, 36] with approximately 3,000 lines of Python.

**Multi-Stage Collaborative Optimizer.** We implement communication orchestration using `batch_isend_and_irecv`, enabling the scattering of gradient subgroups and gathering of updated parameter subgroups within designated CUDA streams. Collaborative computations are orchestrated sequentially on the default CUDA stream to avoid interference. Additionally, we employ two separate CUDA streams to handle host-to-device and device-to-host GPU-CPU communication, further improving optimizer performance. To ensure correct data flow during the optimizer step and pipeline parallelism, we insert appropriate CUDA events; these events enforce communication integrity by verifying that required data is ready before computation begins.

**Crimson Profiler and Planner.** The Profiler is implemented using CUDA events and various CUDA APIs to collect hardware information and runtime status. It also measures coefficients for the auxiliary functions used in scheduling. The Planner formulates and solves the MILP using the high-performance commercial solver Gurobi [8] to efficiently generate the optimal parameter updates strategy.

# 9 Evaluation

## 9.1 Experimental Setup

**Platform.** We evaluate Crimson on 4 training nodes, each equipped with 8 H800 GPUs (80 GB each) interconnected via NVLink, providing up to 400 GB/s of bidirectional bandwidth. The nodes are connected through 400 Gb/s NDR InfiniBand. Each node is further equipped with two Intel Xeon Platinum 8480CL processors with AVX-512 support and a total of 224 threads. GPU–CPU connections are established through PCIe 5.0 x16 links. This configuration represents a practical and cost-accessible setup commonly employed by SMEs for training and fine-tuning LLMs.

**Model, Datasets and Workloads.** We assess Crimson on multiple model sizes derived from the widely used open-source Llama-3 [7] and GPT-3 implementations in Megatron-LM. For consistency, we use the C4 (Colossal Clean Crawled Corpus) pre-training dataset [30], preprocessed before training and then streamed to the models. The sequence length is fixed at 4096, the micro-batch size at 1, and the global batch size at 64. Training employs the Adam optimizer ($\kappa_{\text{opt}} = 6$), following standard multi-node LLM training practice.

**Baselines.** To ensure generality, we evaluate three widely adopted pipeline schedulings: 1F1B [5], Interleaved 1F1B [23], and Zero Bubble [29], all of which are commonly used in LLMs training frameworks. For comparison, we combine these schedulings with several memory-saving techniques:

- `Recomputation` [3]: Reduces memory footprint by discarding forward activations and recomputing them during the backward in training.

- `ZeRO-Offload` [34]: Relieves memory constraints by fully offloading optimizer states to the CPU.
- `ZeRO-Offload++` [42]: Enhances `ZeRO-Offload` by partially offloading optimizer states to the CPU.

**3D Parallelism and Communication Topology.** We evaluate a range of parallelism configurations defined as *(dp, tp, pp)*, where *dp* denotes data parallelism (implemented with ZeRO-1 [31]), *tp* represents tensor parallelism, and *pp* indicates pipeline parallelism. In 3D parallelism configurations under the given hardware setup, tensor and data parallelism are confined to intra-node communication via NVLink, whereas pipeline parallelism involves both intra-node NVLink (only when $pp = 8$) and inter-node InfiniBand connections. In Crimson, however, each stage need to communicate with all other stages to collaboratively update parameters, resulting in most inter-GPU communication during parameter updates being carried out over InfiniBand.

**Model Derivation and Size Scaling.** Unless otherwise specified, the evaluated models are derived from standard model families and follow the same architecture-family and system-setup conventions, while being scaled to match our hardware budgets. To study system behavior under a fixed resource budget, we first identify the largest configuration that fits the memory budget, and then construct smaller variants by removing layers while keeping the remaining runtime setup unchanged as much as possible. This setup isolates system-level effects of model size from unrelated changes in model family or deployment configuration, and avoids artifacts tied to any particular model size.

### 9.2 End-to-End Performance

We evaluate the throughput (TFLOP/s) of Crimson and baselines using Llama-3 and GPT-3 models of various sizes under different parallelism configurations. Throughput speedups, relative to Recomputation (when executable) or ZeRO-Offload, are reported in each column of Figure 10. For each parallelism configuration, we begin with the largest model that ZeRO-Offload can execute within GPU memory limits and then reduce the model size by removing 8 layers.

Across all configurations and pipeline schedulings, Recomputation and ZeRO-Offload consistently deliver the lowest performance, with ZeRO-Offload occasionally performing even worse. Recomputation incurs approximately 33% overhead due to the extra forward required during backward, while ZeRO-Offload is bottlenecked by the CPU's limited compute capacity. Although ZeRO-Offload++ alleviates this by exploiting residual GPU memory in the first stage, its improvements remain modest and diminish as model size increases, since the first stage—typically the most memory-constrained—offers little available memory.

By contrast, Crimson achieves an average speedup of more than 25%, reaching up to 33% in some cases, by efficiently coordinating GPU and CPU resources across all stages. Below, we analyze its performance under specific parallelism settings and schedulings.

**(*dp, tp, pp*) = (2, 2, 8).** In this 3D parallelism setting, we evaluate smaller Llama-3 and GPT-3 models because ZeRO-1 incurs parameter and gradient redundancy, which can trigger OOM errors for larger models at this scale. While ZeRO-2 and ZeRO-3 avoid this redundancy, we do not evaluate ZeRO-2/3 in this pipeline-parallel setting due to additional integration constraints and engineering complexity in our training stack. To make the interaction with other parallel communications explicit, Crimson's optimizer-update transfers are confined to the update phase (after the backward) and are issued between the data-parallel gradient all-reduce and the subsequent parameter all-gather, so they do not introduce new communication dependencies on the forward/backward critical path; moreover, since *tp* and *pp* communications occur only during forward and backward, Crimson's transfers are naturally decoupled from them, and they are scheduled so as not to block the *dp* all-reduce/all-gather operations themselves. Crimson yields up to a 31% performance improvement in this practical configuration.

**(*dp, tp, pp*) = (2, 4, 4).** This setting primarily evaluates performance with four pipeline stages. The *tp* value of 4 introduces heavier communication during forward and backward, reducing GPU utilization. The longer forward and backward durations, however, allow Crimson to achieve up to a 1.33× speedup over these baselines in most cases. Despite fewer stages, Crimson still provides substantial performance gains.

**(*dp, tp, pp*) = (1, 4, 8).** We reduce *dp* to 1 and increase *tp* to 4, eliminating parameter and gradient redundancy and enabling evaluation of larger models. With 8 pipeline stages and no redundancy, collaborative parameter updates fully exploit computational resources and residual memory in later stages, accelerating training. This configuration achieves greater improvements than the previous two.

**With different schedulings.** Under the widely adopted 1F1B scheduling, Crimson consistently delivers substantial performance gains. Interleaved 1F1B requires storing more micro-batches activations, which reduces the maximum model size compared with 1F1B and Zero Bubble, but delivers similar improvements due to its scheduling similarity to 1F1B. In contrast, Zero Bubble stores more tensors for delayed weight backward, leaving less memory available in later stages—particularly in GPT-3 models, where activation backward releases only a small fraction of activations. As a result, combining Zero Bubble with Crimson may yield limited improvements in some cases. Nevertheless, in most scenarios—especially for Llama-3 models—the combination proves effective, achieving notable performance gains.

### 9.3 Memory Footprints Analysis

We measured peak GPU memory usage during training by profiling memory footprints across pipeline stages for the

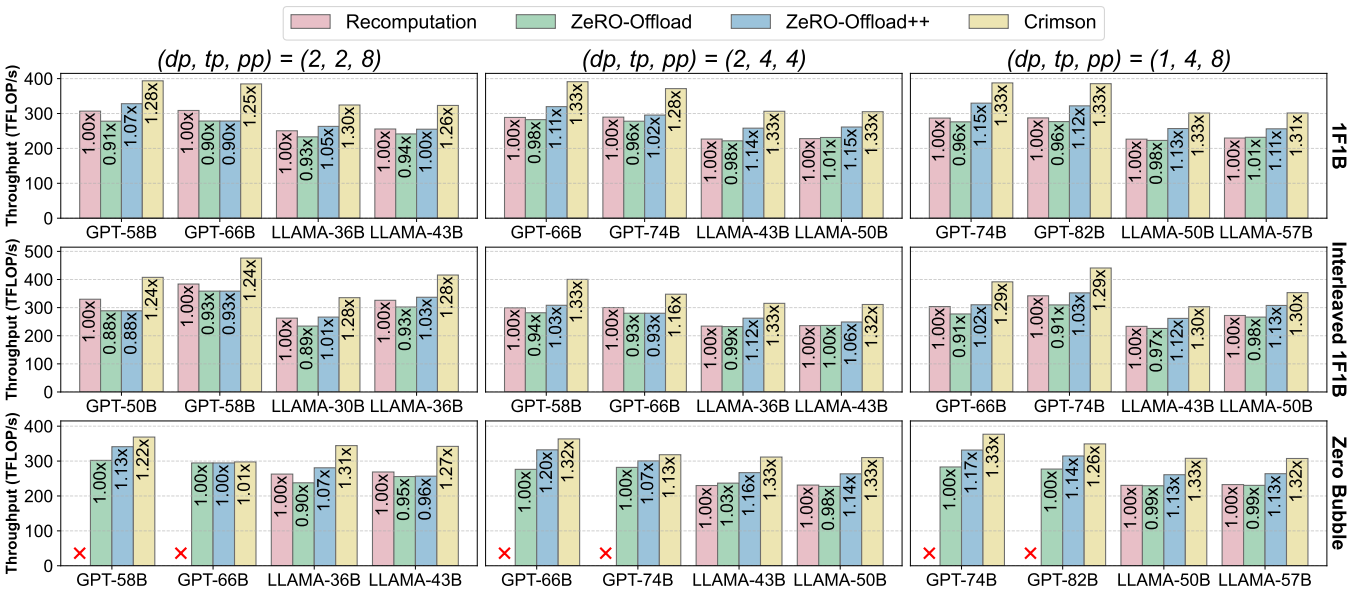



**Figure 10.** End-to-end performance of Llama-3 and GPT-3 models across different parallelism configurations and schedulings.

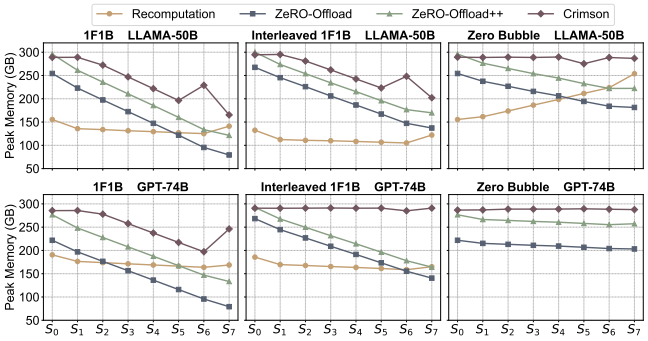



**Figure 11.** Memory footprints across all stages for Llama-3 50B and GPT-3 74B under the (1, 4, 8) parallelism configuration. $S_0$ to $S_7$ indicate pipeline stage IDs.

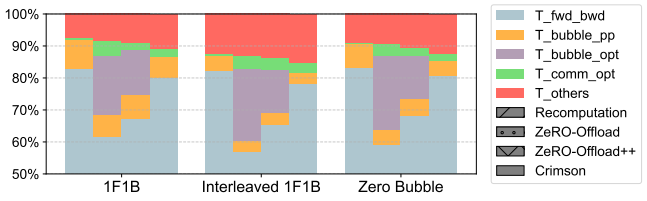



**Figure 12.** Average breakdown of GPU communication and computation under different schedulings with the (1, 4, 8) parallelism configuration on Llama-3 50B.

Llama-3 50B and GPT-3 74B models under the (1, 4, 8) parallelism configuration. As shown in Figure 11, each stage can access a total GPU memory capacity of 320 GB when using four GPUs with tensor parallelism.

Recomputation removes the need to store activations, yielding more balanced memory usage under (Interleaved) 1F1B. Stages 0 and 7 exhibit higher memory usage than the intermediate stages due to the embedding layer at the first stage and the head layer at the last. In contrast, Zero Bubble introduces the opposite imbalance because of storing tensors for delayed weight backward. Both ZeRO-Offload and ZeRO-Offload++ show low GPU memory utilization in later stages and suffer from severe imbalance: under (Interleaved) 1F1B, the most memory-intensive stage requires 1.76× to 3.21× more memory than the least intensive stage, while under Zero Bubble the imbalance ranges from 1.08× to 1.41×.

By comparison, Crimson achieves high GPU memory utilization in most cases—up to 91% under extreme memory constraints—by leveraging residual memory across stages through trapezoid bubble aware scheduling with $\tau = 0.9$. This approach uses available memory in later stages to accelerate parameter updates and eliminates imbalance even under extreme constraints. Overall, Crimson substantially improves GPU memory efficiency relative to baselines.

### 9.4 Communication & Computation Analysis

To assess the effectiveness of Crimson in parameter updates, we profiled GPU runtime across all stages and measured the following components: $T_{\text{fwd\_bwd}}$, $T_{\text{bubble\_pp}}$, and $T_{\text{bubble\_opt}}$, representing the time of forward–backward computation, intrinsic pipeline bubbles, and bubbles introduced by the CPU-based optimizer, respectively. $T_{\text{comm\_opt}}$ denotes the communication overhead during parameter updates—excluding portions overlapped with computation or hidden within bubbles—including both GPU–CPU and GPU–GPU communication, which are not distinguished because of their substantial overlap. Finally, $T_{\text{others}}$ captures additional overhead, such as

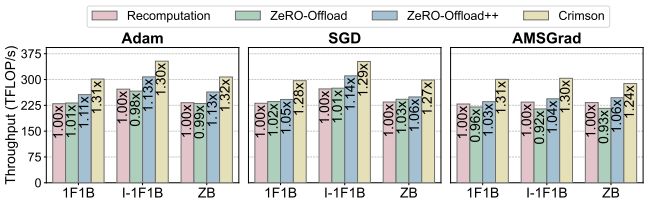



**Figure 13.** Performance across different optimizers with the (1, 4, 8) parallelism configuration on Llama-3 43B (AMSGrad), 50B (Adam), and 57B (SGD). Here, I-1F1B and ZB denote Interleaved 1F1B and Zero Bubble, respectively.

micro-batch loading and intrinsic pipeline communication. All results are reported as percentages in Figure 12.

Among the baselines, Recomputation exhibits the largest $T_{\text{fwd\_bwd}}$ due to the additional forward required during backward computation, while $T_{\text{bubble\_pp}}$ and $T_{\text{bubble\_opt}}$ remain small, indicating efficient use of GPU resources. In contrast, ZeRO-Offload performs poorly, as reflected by its large $T_{\text{bubble\_opt}}$, caused by delays in waiting for CPU-based parameter updates, which reduce GPU utilization. ZeRO-Offload++ shows similar limitations, constrained by the small amount of residual GPU memory in the first stage. Moreover, $T_{\text{comm\_opt}}$ constitutes a significant portion in both ZeRO-Offload and ZeRO-Offload++, since the required data transfers cannot be effectively overlapped with computation.

Crimson fully exploits GPU computational resources, as evidenced by its $T_{\text{fwd\_bwd}}$ and $T_{\text{bubble\_opt}}$ in Figure 12. Across all experiments, Crimson achieves $T_{\text{fwd\_bwd}}$ comparable to Recomputation, reaching over 80%. The observed ~5% $T_{\text{comm\_opt}}$ indicates that Crimson efficiently orchestrates computation and communication within the trapezoid bubble, including CPU–GPU and GPU–GPU communications. Overall, Crimson enhances GPU computational efficiency during collaborative parameter updates, while maintaining well-coordinated computation and communication.

## 9.5 Applicability to Different Optimizers

Besides the Adam [14] ($\kappa_{\text{opt}} = 6$), we evaluate Crimson with SGD [39] ($\kappa_{\text{opt}} = 4$) and AMSGrad [33] ($\kappa_{\text{opt}} = 8$) to demonstrate its applicability across different optimizers.

**9.5.1 Performance Validation.** We evaluate Llama-3 models of different sizes to account for variation in optimizer state size determined by $\kappa_{\text{opt}}$, thereby validating performance improvements under different optimizer algorithms. As shown in Figure 13, compared with Adam—the most widely used optimizer in LLMs training—SGD has smaller optimizer states, enabling support for larger models but restricting the search space in trapezoid bubble aware scheduling, resulting in a maximum speedup of 1.29×. In contrast, AMSGrad, with larger optimizer states, achieves over 1.3× speedup due to its broader search space, though it may face limited available memory (in Zero Bubble) that limit further performance

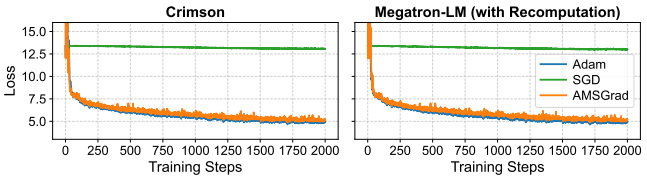



**Figure 14.** Loss curves of Llama-3 using Megatron-LM (with Recomputation) and Crimson under different optimizers.

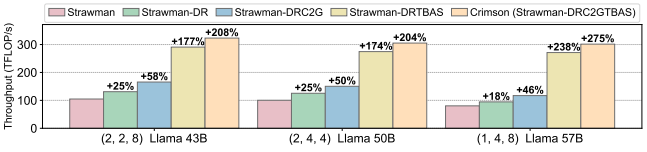



**Figure 15.** Ablation study under 1F1B scheduling with different Llama-3 model sizes and parallelism configurations.

gains. Overall, Crimson delivers an average performance improvement of over 25% across these optimizers, demonstrating strong applicability to diverse optimization algorithms.

**9.5.2 Convergence Validation.** We further examine the impact of Crimson on the convergence behavior of Llama-3. Figure 14 presents the training loss over 2,000 iterations using Megatron-LM (with Recomputation) and Crimson with three different optimizers. With SGD, the loss decreases more slowly, reflecting the inherent inefficiency of applying SGD in LLMs training. In contrast, both Adam and AMSGrad yield rapid convergence to a loss of approximately 6 within the first 500 iterations, followed by a gradual decline and eventual stabilization around 5. The loss curves for Megatron-LM and Crimson are nearly identical, confirming that Crimson preserves convergence behavior while ensuring training correctness. These results highlight its suitability for real-world large-scale LLMs training scenarios.

## 9.6 Ablation Study

To assess the contribution of individual components in Crimson, we evaluate several variants under different parallelism configurations and model sizes, as illustrated in Figure 15. The baseline, `Strawman`, implements collaborative parameter updates (subsection 3.4, Figure 8a) with update tasks evenly distributed across subsequent stages following each preceding stage. `Strawman-DR` extends this by decomposition of scattering-gathering and reordering of computation sequences (subsection 5.2). `Strawman-DRC2G` further incorporates dynamic loading and evicting of optimizer states (subsection 5.2). `Strawman-DRTBAS` adds trapezoid bubble aware scheduling (section 6). The complete system, Crimson (`Strawman-DRC2GTBAS`), integrates all components.

As shown in Figure 15, comparing Strawman with Strawman-DR demonstrates that eliminating all-to-all communication and coordinating computations yield consistent performance

improvements. The comparison between Strawman and Strawman-DRC2G shows that dynamic loading and evictingof optimizer states further enhance performance. Similarly, comparing Strawman with Strawman-DRTBAS highlights that trapezoid bubble aware scheduling effectively leverages both GPU and CPU resources across all stages, leading to significant performance gains. Finally, the comparison of Strawman-DRTBAS and Strawman-DRC2GTBAS confirms that the benefits of dynamic loading and evictingof optimizer states persist when combined with trapezoid bubble aware scheduling. Collectively, these components demonstrate that Crimson achieves an efficient system design by systematically enhancing collaborative parameter updates.

## 10 Related Works

**Memory-Saving Techniques.** Numerous studies [9, 28, 32, 35, 43] have investigated memory optimization for large-scale model training. TMOF [18] reduces PCIe contention by coordinating tensor movements through disjoint swapping and bidirectional overlapping. However, these approaches rely heavily on frequent GPU–CPU swaps, which limits their applicability to modern hardware.

BPipe [13] addresses memory imbalances by transferring activations between stages to balance GPU memory usage, but introduces additional communication overhead and is poorly suited for tensor parallelism. MPress [48] extends model capacity by combining fast inter-GPU swaps, recomputation, and GPU–CPU swaps, though it faces challenges in scaling across multiple nodes. AdaPipe [37] jointly optimizes memory and computation through selective recomputation and dynamic workload balancing across stages, making it most effective for large-scale training with many nodes and long sequences. Other studies [38, 41, 49] further integrate memory-saving methods into pipeline parallelism, aiming to reduce memory footprints while sustaining throughput. Overall, these systems primarily focus on improving memory-efficiency trade-offs so that training remains feasible under limited GPU memory budgets.

In contrast, Crimson targets a different bottleneck and is complementary to prior memory-saving methods such as (selective) recomputation (e.g., AdaPipe [37] and MIST [49]) and ZeRO-style state partitioning (e.g., ZeRO-3 [31]). While those techniques mainly reduce peak GPU memory usage, Crimson optimizes the optimizer-update phase under pipeline training with offloading, which can still introduce substantial GPU idleness and iteration-time overhead even after memory capacity is no longer the primary constraint. Crimson therefore does not replace these methods; instead, it improves update-phase efficiency by coordinating parameter transfers and updates across pipeline stages to reduce idle time on the critical path. As a result, Crimson is most beneficial when offloaded updates are non-trivial and stage-level idleness is visible, while its gains are naturally smaller when the model and optimizer states already fit comfortably in GPU memory and update costs are fully hidden.

**Distributed Training with Parallelism.** Distributed training across multiple devices [1, 4, 19, 40, 47] is enabled by a range of parallelism strategies. ZeRO [31] and FSDP [46] exploit data parallelism by partitioning large batches into smaller mini-batches processed independently across GPUs. Megatron-LM [15, 23, 36] introduces tensor parallelism by splitting tensors into sub-tensors distributed across GPUs for concurrent computation. Furthermore, Megatron-LM, Ring Attention [20], and DeepSpeed-Ulysses [11] propose sequence parallelism, distributing long-sequence processing across multiple devices. Collectively, these strategies distribute workloads across GPUs and nodes to accelerate training. By contrast, Crimson emphasizes reducing GPU memory consumption while preserving near-equivalent training performance, primarily serving as an update-phase efficiency technique for pipeline training with optimizer offloading.

## 11 Conclusion

This paper introduces Crimson, an efficient system that integrates collaborative parameter updates—featuring a multi-stage collaborative optimizer and trapezoid bubble aware scheduling—to distribute update tasks across stages. These designs enhance training performance, increases GPU utilization, and provides a practical solution for LLMs training.

## Acknowledgements

The work described in this paper was supported in part by the Major Key Project of PCL under Grant No. PCL2025A03, the National Natural Science Foundation of China under Grant No. 62472459, and the Guangdong S&T Program under Grant No. 2025B0101080001. This research was also supported by the Hong Kong RGC General Research Fund under Grants No. 152228/23E, No. 162161/24E, No. 162116/25E, and No. 162180/25E; the National Natural Science Foundation of China Key Program under Grant No. 62532005; the Collaborative Research Fund under Grant No. C1042-23GF and No. 5097-25G; the NSFC/RGC Collaborative Research Scheme under Grant No. 62461160332 and No. CRS_HKUST602/24; the Research Impact Fund under Grant No. R5011-23; the Areas of Excellence Scheme under Grant No. AoE/E-601/22-R; and the InnoHK (HKGAI).

## References

[1] Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. 2022. Varuna: scalable, low-cost training of massive deep learning models. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 472–487.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess,

Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.

[3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).

[4] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. 2024. Parcae: Proactive,{Liveput-Optimized}{DNN} Training on Preemptible Instances. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1121–1139.

[5] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. 2021. DAPPLE: A pipelined data parallel approach for training large models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 431–445.

[6] Jiarui Fang, Zilin Zhu, Shenggui Li, Hui Su, Yang Yu, Jie Zhou, and Yang You. 2022. Parallel training of pre-trained models via chunk-based dynamic memory management. *IEEE Transactions on Parallel and Distributed Systems* 34, 1 (2022), 304–315.

[7] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[8] Gurobi Optimization, LLC. 2024. Gurobi Optimizer Reference Manual. https://www.gurobi.com

[9] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1341–1355.

[10] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).

[11] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509* (2023).

[12] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).

[13] Taebum Kim, Hyoungjoo Kim, Gyeong-In Yu, and Byung-Gon Chun. 2023. BPIPE: memory-balanced pipeline parallelism for training large language models. In *International Conference on Machine Learning*. PMLR, 16639–16653.

[14] Diederik P Kingma. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

[15] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023), 341–353.

[16] Shenggui Li, Hongxin Liu, Zhengda Bian, Jiarui Fang, Haichen Huang, Yuliang Liu, Boxiang Wang, and Yang You. 2023. Colossal-ai: A unified deep learning system for large-scale parallel training. In *Proceedings of the 52nd International Conference on Parallel Processing*. 766–775.

[17] Zhenxing Li, Qiang Cao, Yajie Chen, and Wenrui Yan. 2023. CoTrain: Efficient Scheduling for Large-Model Training upon GPU and CPU in Parallel. In *Proceedings of the 52nd International Conference on Parallel Processing*. 92–101.

[18] Shao-Fu Lin, Yi-Jung Chen, Hsiang-Yun Cheng, and Chia-Lin Yang. 2023. Tensor Movement Orchestration in Multi-GPU Training Systems. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1140–1152.

[19] Guodong Liu, Youshan Miao, Zhiqi Lin, Xiaoxiang Shi, Saeed Maleki, Fan Yang, Yungang Bao, and Sa Wang. 2024. Aceso: Efficient Parallel DNN Training through Iterative Bottleneck Alleviation. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 163–181.

[20] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889* (2023).

[21] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2017. Mixed precision training. *arXiv preprint arXiv:1710.03740* (2017).

[22] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.

[23] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.

[24] NVIDIA. 2024. *NVLink & NVSwitch: Fastest HPC Data Center Platform*. https://www.nvidia.com/en-us/data-center/nvlink/

[25] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. 2013. On the difficulty of training recurrent neural networks. In *International conference on machine learning*. Pmlr, 1310–1318.

[26] A Paszke. 2019. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703* (2019).

[27] Houwen Peng, Kan Wu, Yixuan Wei, Guoshuai Zhao, Yuxiang Yang, Ze Liu, Yifan Xiong, Ziyue Yang, Bolin Ni, Jingcheng Hu, et al. 2023. Fp8-lm: Training fp8 large language models. *arXiv preprint arXiv:2310.18313* (2023).

[28] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based gpu memory management for deep learning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 891–905.

[29] Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. 2024. Zero Bubble (Almost) Pipeline Parallelism. In *The Twelfth International Conference on Learning Representations*.

[30] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67. http://jmlr.org/papers/v21/20-074.html

[31] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.

[32] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–14.

[33] Sashank J Reddi, Satyen Kale, and Sanjiv Kumar. 2019. On the convergence of adam and beyond. *arXiv preprint arXiv:1904.09237* (2019).

[34] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. {Zero-offload}: Democratizing {billion-scale} model training. In

*2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 551–564.

[35] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W Keckler. 2016. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–13.

[36] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[37] Zhenbo Sun, Huanqi Cao, Yuanwei Wang, Guanyu Feng, Shengqi Chen, Haojie Wang, and Wenguang Chen. 2024. AdaPipe: Optimizing Pipeline Parallelism with Adaptive Recomputation and Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 86–100.

[38] Zhenbo Sun, Shengqi Chen, Yuanwei Wang, Jian Sha, Guanyu Feng, and Wenguang Chen. 2025. MEPipe: Democratizing LLM Training with Memory-Efficient Slice-Level Pipeline Scheduling on Cost-Effective Accelerators. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1263–1278.

[39] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. 2013. On the importance of initialization and momentum in deep learning. In *International conference on machine learning*. pmlr, 1139–1147.

[40] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Pat McCormick, Jamaludin Mohd-Yusof, et al. 2022. Unity: Accelerating {DNN} training through joint optimization of algebraic transformations and parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 267–284.

[41] Xinyi Wan, Penghui Qi, Guangxing Huang, Jialin Li, and Min Lin. 2025. PipeOffload: Improving Scalability of Pipeline Parallelism with Memory Optimization. *arXiv preprint arXiv:2503.01328* (2025).

[42] Guanhua Wang, Masahiro Tanaka, Xiaoxia Wu, Lok Chand Koppaka, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He (team lead). 2023. *DeepSpeed ZeRO-Offload++: 6x Higher Training Throughput via Collaborative CPU/GPU Twin-Flow*. https://github.com/microsoft/DeepSpeed/blob/master/blogs/deepspeed-offloadpp/README.md

[43] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming*. 41–53.

[44] wikipedia. 2024. *InfiniBand from Wikipedia, the free encyclopedia*. https://en.wikipedia.org/wiki/InfiniBand

[45] wikipedia. 2024. *PCI Express from Wikipedia, the free encyclopedia*. https://en.wikipedia.org/wiki/PCI_Express

[46] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277* (2023).

[47] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.

[48] Quan Zhou, Haiquan Wang, Xiaoyan Yu, Cheng Li, Youhui Bai, Feng Yan, and Yinlong Xu. 2023. Mpress: Democratizing billion-scale model training on multi-gpu servers via memory-saving inter-operator parallelism. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 556–569.

[49] Zhanda Zhu, Christina Giannoula, Muralidhar Andoorveedu, Qidong Su, Karttikeya Mangalam, Bojian Zheng, and Gennady Pekhimenko. 2025. Mist: Efficient Distributed Training of Large Language Models via Memory-Parallelism Co-Optimization. In *Proceedings of the Twentieth European Conference on Computer Systems*. 1298–1316.